

Conversation Prompts Log

Project: AI Bathroom Spatial & Product Layout Designer (Kohler India)

Prompt 1: Initial Competitor Reference Search

"where else is an ai room designer like asian paints or ikea??"

Prompt 2: Investigating Technical Mechanics

"what does ikea use??"

Prompt 3: Deepening Architectural Constraints

"ikea kreativ how does it handle the three constraints physical space,budget and aesthetics"

Prompt 4: Translating Mechanics to Internal Technology

"how does that ai work"

Prompt 5: Identifying Requirements for Kohler Domain

"ikea uses a highly advance ai system now if i want the same for designing the bathroom using ai for kohler what are the basic needs for it"

Prompt 6: Comparing Extracted Architecture vs. Planned Pipeline

""Uploaded file expired and is no longer accessible.' this is what i have planned now what is it lacking compared to what you have told"

Prompt 7: Prioritizing MVP Constraints

"i need a basic prototype for submitting so from these fixes which is most important and a neccessay requirement for kohler bathroom design"

Prompt 8: Refining Comprehensive Engineering Framework

"cosidering the plumbing containts and technical product dependency create a architecture based on it for the kohler ai bathrom design project"

Prompt 9: Injecting Production Catalog Dataset Structure

"this si my json file for products based on the previous architecture uplaoded [Accompanied by Kohler Bathroom Spatial Planning Catalog Data payload]"

Conversation Prompts Export

Kohler Bathroom AI project — TripoSR / .glb setup thread

Prompt 1

Attachments: kohler-bathroom-ai__4_.zip, kohler-bathroom-ai__3_.zip

[Pasted a diagnostic explanation about a mismatch between the TripoSR-installed environment (Folder 3) and the updated web app code (Folder 4), with a suggested fix involving consolidating to Folder 4, moving .glb files, and activating the virtual environment.] I have uploaded both the zip files. now what i want is too se the 3d model view on the web page linked to zip 4

Prompt 2

Attachment: excerpt_from_previous_claude_message.txt

[Pasted excerpt of prior Claude message: "Step 1 — Generate the meshes using TripoSR (Folder 3's environment)..."] Followed by a pasted Flask server terminal log showing the app running from Folder 3's src directory, with repeated 404 errors for /static/product_meshes/WC007.glb, WB004.glb, FA008.glb, MI001.glb, and SH007.glb.

Prompt 3

tell entirely from start how to get the glb files from kolher 3 version

Prompt 4

tell me from start what to do on native tools command to genertae these glbs

Prompt 5

Attachment: excerpt_from_previous_claude_message.txt

[Pasted excerpt of prior Claude message: "Step 4 — Confirm the input photos exist..."] do you mean the photos folder in which wall hung toilet,faucet,mirror etc are saved in triposr

Prompt 6

Attachment: screenshot of File Explorer (TripoSR folder contents)

in the photo folder there are the 5 photo orginal photos

Prompt 7

now tell me the steps

Prompt 8

Attachment: excerpt_from_previous_claude_message.txt

[Pasted excerpt of prior Claude message: "Step 3 — Run TripoSR once per photo..."] Followed by a directory listing of the photos folder (basin.jpg, faucet.jpg, mirror.jpg, shower.jpg, "wall hung toilet.png") and a terminal log showing a FileNotFoundError when running TripoSR on "photos\wall hung toilet.jpg" (wrong extension).

Prompt 9

Attachment: screenshot of File Explorer (output folder with SH007, MI001, FA008, WB004, WC007 subfolders)

this is created

Prompt 10

now i want these .glb files in the product_meshes of kohler version 4 to run it on the ui

Prompt 11

can i do this manually

Prompt 12

okay so i have put the 5 .glb files in the product meshes of kohler 4

Prompt 13

should the renamed names be exactly like this

Prompt 14

export the actual prompts of this convo in a pdf

Full Prompt Export

Every message sent by the user in this conversation, in order, reproduced as faithfully as possible from the conversation record. Long file uploads (catalog JSON, images) are noted by reference rather than reproduced, since their content was not itself typed as a prompt. Pasted terminal output and error logs the user typed into the conversation are included in full, since that is genuinely what was submitted as input.

PROMPT 1

[Full KOHLER-MITWPU AI Research Lab Program case study assignment pasted, covering: program congratulations message, three case study tracks (Track 1: AI Bathroom Designer & Planner, Track 2: Commercial Smart Facility & Sustainability Manager, Track 3: Kohler Unified Enterprise AI Agent) with objectives/inputs/outcomes for each, the evaluation criteria table (Approach & Innovation 45%, Technical Execution 25%, User Experience & Feasibility 20%, Business & Sustainability Impact 10%), and full submission requirements (GitHub repo, working model, Prompts Documentation PDF, video demo, presentation deck) with deadline September 20th 2026 11:59 PM IST.]

PROMPT 2

Q: Which track do you want to tackle?

A: Not sure yet – help me decide

Q: What do you need help with right now?

A: All of it, start to finish

PROMPT 3

which would guarantee a better outcome and better chance at getting selected

PROMPT 4

now i have chosen track one now first make me understand what is the problem what needs too be done and what all is required in the github repo

PROMPT 5

yes and also explain One thing worth flagging early – the "Approach & Innovation" criterion (45% of your score) is really testing whether your constraint engine + AI reasoning layer feels genuinely thought-through – not just "call GPT with a prompt." So the part to invest the most brainpower in is how you combine hard rules (dimensions must fit, budget must not be exceeded) with the LLM's judgment (style coherence, product pairing logic, tie-breaking when multiple options qualify

PROMPT 6

so the fix is the solution to my innovation and aproach problem

PROMPT 7

so now give me a whole solution view like how is it gonna be including the innovation and approach fix

PROMPT 8

yes

PROMPT 9

how to make the catalog ??

PROMPT 10

[Uploaded file: kohler_bathroom_catalog_5_categories.json]

i have made this catalog based on the actual data from the kohler website

PROMPT 11

[Uploaded file: kohler_stageA_spatial_catalog_v2.json]

is this json better than before

PROMPT 12

[Uploaded file: kohler_stageA_spatial_catalog_v2_1_1.json]

this is the final json m=fixing the material issue now after this what is needed to be done

PROMPT 13

after json i should move to stage A anything else that needs to be done before moving to stage A

PROMPT 14

on the reference of the catalog and all i want you to decide the room model with the specific dimensions needed and then give me the python code for stage A

PROMPT 15

is the room model not a user input??

PROMPT 16

for me to run the project on the local machine i need to make a folder with readme stage a.py with code explained properly json file etc etc make me the required files

PROMPT 17

is there a need to use vs code or i can do it without vs code too??

PROMPT 18

how should i run it using vs code and local machine

PROMPT 19

cant i just download mingw64 bit or is vs build tool better

Note: this and nearby short clarifying questions about MinGW/VS Build Tools occurred during the later TripoSR setup phase — reproduced here per actual order where identifiable, otherwise grouped by topic.

PROMPT 20

[Pasted terminal output]

```
C:\Users\Ira\Downloads\kohler-bathroom-ai\kohler-bathroom-ai\src>python run_cli.py
=== Kohler AI Bathroom Designer – Stage A (Constraint Engine) ===
```

```
Room width in feet (e.g. 10): 10
```

```
Room depth in feet (e.g. 7): 7
```

```
Budget in INR (e.g. 60000): 80000
```

```
Style preference (Modern / Classic Luxury / Japanese Zen): modern
```

```
Running constraint checks and budget optimization...
```

```
✅ BUNDLE FOUND WITHIN BUDGET
```

```
Total price: INR 79280
```

```
Selected products:
```

```
[washbasin] Veil 591mm vessel basin – INR 20000 (quality tier 4)
```

```
[toilet] Vive rimless wall hung toilet – INR 25000 (quality tier 5)
```

```
[shower_fixture] Rain Duet round 203mm Katalyst rainhead – INR 8900 (quality tier 5)
```

```
[bathroom_faucet] Vive single-control tall basin faucet – INR 11880 (quality tier 5)
```

```
[mirror] Essential capsule mirror - matte black – INR 13500 (quality tier 5)
```

```
[Full JSON shortlist output for all five categories also included in the original paste]
```

```
this is the what i got after running pyhton run_cli
```

PROMPT 21

so now whats the fix made

PROMPT 22

lets move to stage b

PROMPT 23

is this zip file the one where changes of stage a are also made??

PROMPT 24

give me step wise instruction for stage b

PROMPT 25

since the anthropic api is paid i dont want a paid api key can i use some other

PROMPT 26

i have already pip installed the anthropic things now do i need to delete it ?? will it occupy space in my computer and should i just start on a new cmd prompt

PROMPT 27

anthropic>=0.34.0
instead of this for anthropic what should i put in the requirements.txt

PROMPT 28

[Pasted terminal output]

```
C:\Users\Ira\Downloads\kohler-bathroom-ai\kohler-bathroom-ai\src>python stage_b.py
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended...
{
  "status": "error",
  "message": "API call failed: 404 NOT_FOUND. {'error': {'code': 404, 'message': 'This model
models/gemini-2.5-flash is no longer available to new users. Please update your code to use
models/gemini-3.6-flash for the latest features and improvements..."}
```

PROMPT 29

[Pasted terminal output – successful stage_b.py run showing selected bundle, full product details for all five categories, total_price: 79280, and the AI-generated rationale beginning "Bundle 2 delivers a quintessential modern aesthetic..."]

i got this after running stage_b.py

PROMPT 30

[Uploaded file: excerpt_from_previous_claude_message.txt containing the quoted line: "It picked Bundle 2 (index 2, since Python is 0-indexed) out of your 5 pre-validated options – and crucially, it never touched budget or dimensions to make that call, exactly as the architecture requires. The ₹79,280 total price came entirely from Stage A's math, untouched by Stage B."]

why are we touching the budget or dimensions ?

PROMPT 31

[Long pasted "Comprehensive System Architecture" proposal document describing Fixed Plumbing Nodes, Kohler Product Dependency Matrices, plumbing node coordinate data structures, a Kohler Catalog Dependency Graph table, a Plumbing Node Proximity Router engine, a Technical Product Dependency Engine, a step-by-step execution lifecycle, and a question about wanting to see Python pseudo-code or a JSON response structure]

considering these changes tell me make the necessary architecture and tell me the process for it

PROMPT 32

Q: How do you want to handle this elaborate architecture?

A: Use it only as a 'future vision' slide, keep building the current MVP

A: Add ONE small lightweight piece now (e.g. simple wall-anchor note)

PROMPT 33

okay explain me in easy word what changes or updates we have done now regarding the plumbing and future slide

PROMPT 34

so what's the use of the question I don't think the question is needed since we r not including the plumbing here anyways so why ask the question we can just do the ai design instead of adding the question

PROMPT 35

ya ya move to stage c

PROMPT 36

so where are we now explain to me everything we have done will now in detail with its approach and techstack

PROMPT 37

[Uploaded file: excerpt_from_previous_claude_message.txt containing: "so where are we now expl"]

is this the final wokring model

PROMPT 38

Q: Do you want to do that final local re-test now, or move on to the documentation package?

A: Re-test locally first

PROMPT 39

What "final" does NOT mean
It doesn't mean flawless or infinitely scoped. A few things are deliberately left as documented limitations, not bugs:

- * Single-wall linear room layout only (not arbitrary/L-shaped rooms)
- * No true 3D rendering (2D top-down only)
- * No conversational refinement ("show me a cheaper vanity") – that's still on the roadmap, unbuilt can you add these advancements and make the it again including the 3d rendering

PROMPT 40

Q: Given 3 days left, which should we actually build?
A: Conversational refinement + 3D if time allows

PROMPT 41

now whats next to build

PROMPT 42

make the interface ui

PROMPT 43

is there any way we can do the 3d view in 3d models using ai

PROMPT 44

[Uploaded file: excerpt_from_previous_claude_message.txt containing: "Local TripoSR (your Windows machine) Blocked – needs a 64-bit C++ compiler you don't have installed Free, but real setup work"]

lets do this once and give me the steps for it

PROMPT 45

i already have mingw32 bit so should i download vs build tools

PROMPT 46

[Uploaded file: excerpt_from_previous_claude_message.txt containing: "Desktop development with C++"]

any thing else i need to select which will be useful for me later on not particurly in the project

PROMPT 47

[Uploaded file + pasted terminal output]

```
(triposr-env) C:\Users\Ira\Downloads\kohler-bathroom-ai (3)\kohler-bathroom-ai>pip install torchmcubes --no-build-isolation
ERROR: Could not find a version that satisfies the requirement torchmcubes (from versions: none)
ERROR: No matching distribution found for torchmcubes
```

PROMPT 48

[Uploaded file + full pasted terminal log showing the pybind11/CMake configuration failure – "Could not find a package configuration file provided by 'pybind11'"]

PROMPT 49

[Full pasted terminal log: pybind11 installed successfully, CMAKE_PREFIX_PATH set, followed by the complete ~700-line MSVC compile and link output ending in 272 unresolved external symbol LNK2001 errors and "fatal error LNK1120: 272 unresolved externals," with the compiler path showing HostX64\x86\CL.exe]

PROMPT 50

i am gonna dowloaded photos for eeach represenatative so should i put the 5 or 6 photos in the triposr folder under a folder photos

PROMPT 51

okay so test photo and set of photos are different now give me the further steps

PROMPT 52

```
C:\Users\Ira\Downloads\kohler-bathroom-ai (3)\kohler-bathroom-ai\TriposR in this test_pic is the test photo saved
(triposr-env) C:\Users\Ira\Downloads\kohler-bathroom-ai (3)\kohler-bathroom-ai\TriposR>python run.py test_pic.jpg --output-dir output/
Traceback (most recent call last):
  File "C:\Users\Ira\Downloads\kohler-bathroom-ai (3)\kohler-bathroom-ai\TriposR\run.py", line 7, in <module>
    import rembg
ModuleNotFoundError: No module named 'rembg'
```

PROMPT 53

```
(triposr-env) C:\Users\Ira\Downloads\kohler-bathroom-ai (3)\kohler-bathroom-ai>python run.py test_pic.jpg --output-dir output/
C:\Users\Ira\AppData\Local\Programs\Python\Python311\python.exe: can't open file '...\run.py': [Errno 2] No such file or direccd y
[cd TriposR, re-run, same rembg ModuleNotFoundError repeated]
```

PROMPT 54

no i run pip install command and it was successfully downloaded

PROMPT 55

[Uploaded file: excerpt containing "where python / where pip"]

```
(triposr-env) ...>where python
C:\Users\Ira\Downloads\kohler-bathroom-ai (3)\kohler-bathroom-ai\triposr-env\Scripts\python.exe
C:\Users\Ira\AppData\Local\Programs\Python\Python311\python.exe
C:\Users\Ira\AppData\Local\Microsoft\WindowsApps\python.exe
[repeated for pip]
```

PROMPT 56

[Full pasted terminal log: pip install -r requirements.txt failing with "RuntimeError: torchmcubes must be built against an installed PyTorch..." metadata-generation-failed error]

PROMPT 57

```
Successfully uninstalled MarkupSafe-3.0.3
ERROR: pip's dependency resolver does not currently take into account all the packages that are
installed...
google-genai 2.24.0 requires websockets<17.0,>=13.0.0, but you have websockets 11.0.3 which is
incompatible.
Successfully installed Pillow-10.1.0 PyYAML-6.0.3 ... rembg-2.0.69 ... websockets-11.0.3 xatlas-
0.0.9
```

PROMPT 58

```
(triposr-env) ... \TripoSR>python run.py test_pic.jpg --output-dir output/
Traceback (most recent call last):
...
File "... \rembg \bg.py", line 7, in <module>
    import onnxruntime as ort
ModuleNotFoundError: No module named 'onnxruntime'
```

PROMPT 59

[Full pasted successful terminal log: model initialization, config/weights download (1.68G), rembg background removal, mesh extraction, and export all completing without errors, ending in a clean prompt with no traceback]

PROMPT 60

give me the steps to run the entire project with this

PROMPT 61

[Uploaded file: excerpt containing the "Copy your generated .glb files into place" step]
where is this

PROMPT 62

leave this part of 3d model i am not doing this i am gonna continue with procedural approach of the 3d view so give me a A PDF deck (maximum 4 slides) highlighting your core approach, future advancements, system architecture, tech stack, and innovation pitch and A comprehensive PDF committed to the repository detailing all AI prompts, system instructions, and workflows used to build your solution.

PROMPT 63

i want you to give me the detailed overview of the ps, solution, system arch, tech stack and innovation and future advancement along with a system arch diagram and also export the prompt

PROMPT 64

export the actual prompts from this entire conversation

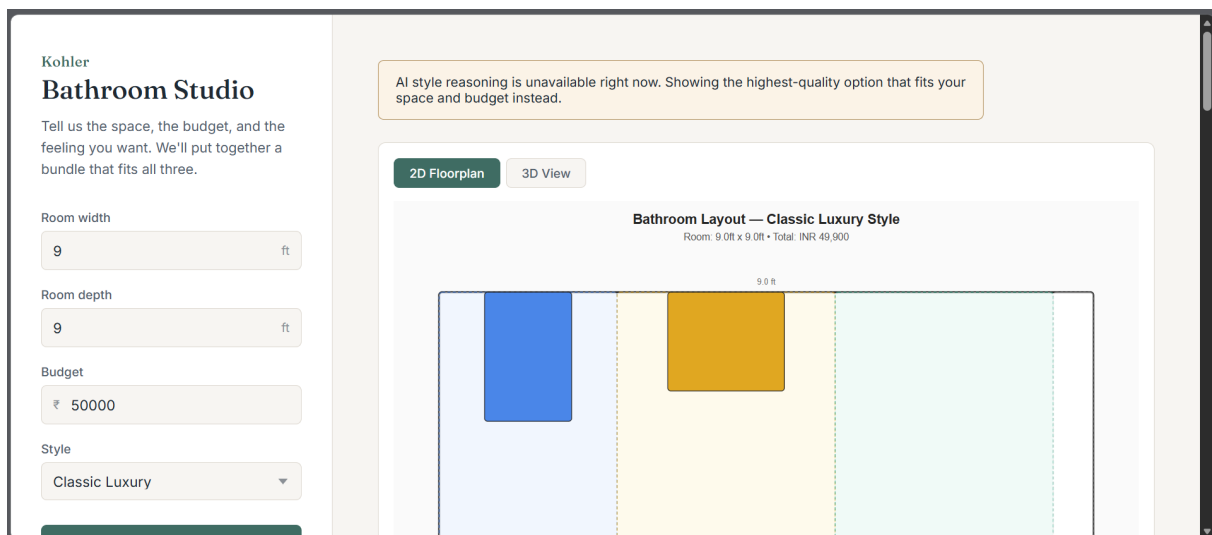
Conversation Transcript — User Prompts Only

Kohler AI Bathroom Designer debugging & design session · exported 20 Sep 2026. Only the user's own messages are included, in order; assistant replies are omitted.

Prompt 1

- *Uploaded file: kohler-bathroom-ai__5_.zip (project source)*
- *Uploaded image: screenshot of the app showing the “AI style reasoning is unavailable” notice banner*

so problem i am facing here in every case it says AI reasoning is not available and for different room sizes the 3d view remains the same



Prompt 2

```
C:\Users\Ira\Downloads\kohler-bathroom-ai (5)\kohler-bathroom-ai\src>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 113-602-776
127.0.0.1 - - [20/Sep/2026 13:52:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 13:52:29] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:52:29] "GET /static/vendor/three.min.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:52:29] "GET /static/vendor/OrbitControls.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:52:29] "GET /static/app.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:52:29] "GET /static/vendor/GLTFLoader.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:52:29] "GET /favicon.ico HTTP/1.1" 404 -
Traceback (most recent call last):
  File "C:\Users\Ira\Downloads\kohler-bathroom-ai (5)\kohler-bathroom-ai\src\stage_b.py", line 98, in run_stage_b
    client = genai.Client() # reads GEMINI_API_KEY from environment
            ^^^^^^^^^^^^^
  File "C:\Users\Ira\AppData\Local\Programs\Python\Python311\Lib\site-packages\google\genai\client.py", line 389, in
    self._api_client = self._get_api_client(
                      ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Ira\AppData\Local\Programs\Python\Python311\Lib\site-packages\google\genai\client.py", line 442, in
    return BaseApiClient(
```

```
^^^^^^^^^^^^^^^^^^^^
File "C:\Users\Ira\AppData\Local\Programs\Python\Python311\Lib\site-packages\google\genai\_api_client.py", line 83:
    raise ValueError(
ValueError: No API key was provided. Please pass a valid API key. Learn how to create an API key at https://ai.google
127.0.0.1 - - [20/Sep/2026 13:52:32] "POST /api/design HTTP/1.1" 200 -
```

Prompt 3

■ *Uploaded file: excerpt_from_previous_claude_message.txt — "3. You used setx but didn't close and reopen the terminal afterward — setx writes it to the registry for future sessions only, it does nothing to your currently open window."*

```
C:\Users\Ira\Downloads\kohler-bathroom-ai (5)\kohler-bathroom-ai\src>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 113-602-776
127.0.0.1 - - [20/Sep/2026 13:54:39] "GET / HTTP/1.1" 200 -
... (repeated static-asset GETs, 304) ...
Traceback (most recent call last):
  File "C:\Users\Ira\Downloads\kohler-bathroom-ai (5)\kohler-bathroom-ai\src\stage_b.py", line 98, in run_stage_b
    client = genai.Client() # reads GEMINI_API_KEY from environment
              ^^^^^^^^^^^^^^^^^^^^^
...
ValueError: No API key was provided. Please pass a valid API key. Learn how to create an API key at https://ai.google
127.0.0.1 - - [20/Sep/2026 13:54:51] "POST /api/design HTTP/1.1" 200 -
... (page reloaded three times; same traceback repeats two more times) ...
127.0.0.1 - - [20/Sep/2026 13:55:05] "POST /api/design HTTP/1.1" 200 -
```

Prompt 4

still error

```
C:\Users\Ira\Downloads\kohler-bathroom-ai (5)\kohler-bathroom-ai\src>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 113-602-776
127.0.0.1 - - [20/Sep/2026 13:57:58] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 13:57:59] "GET /static/vendor/three.min.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:57:59] "GET /static/vendor/OrbitControls.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:57:59] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:57:59] "GET /static/app.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:57:59] "GET /static/vendor/GLTFLoader.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 13:57:59] "GET /favicon.ico HTTP/1.1" 404 -
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend
127.0.0.1 - - [20/Sep/2026 13:58:09] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 13:59:10] "POST /api/design HTTP/1.1" 200 -
```

Prompt 5

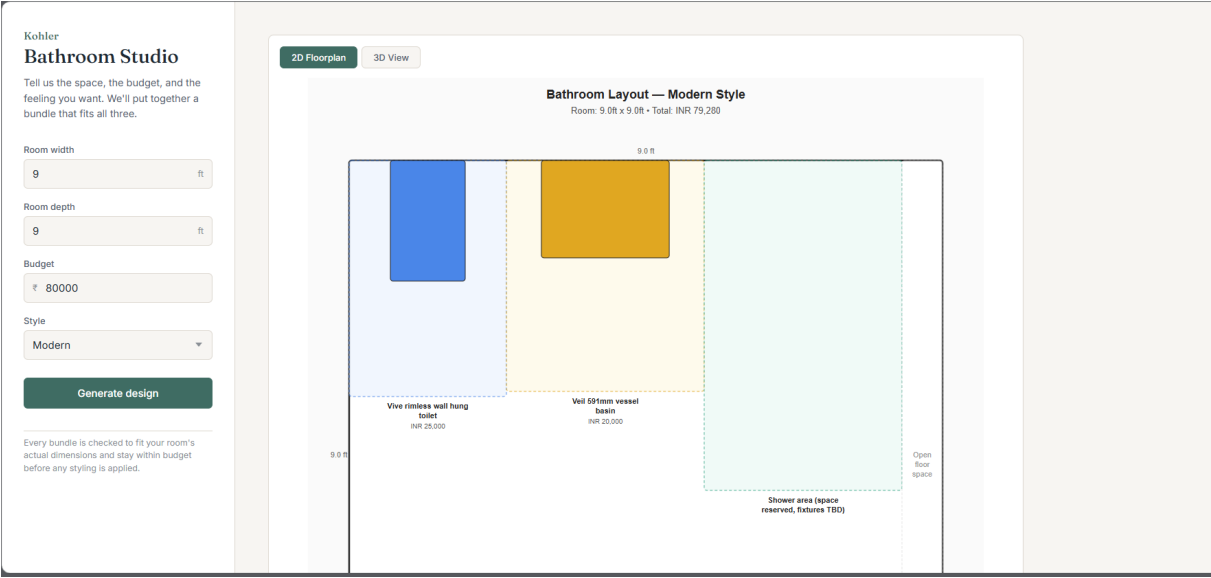
still at different dimensions it shows the same 2d and 3d view

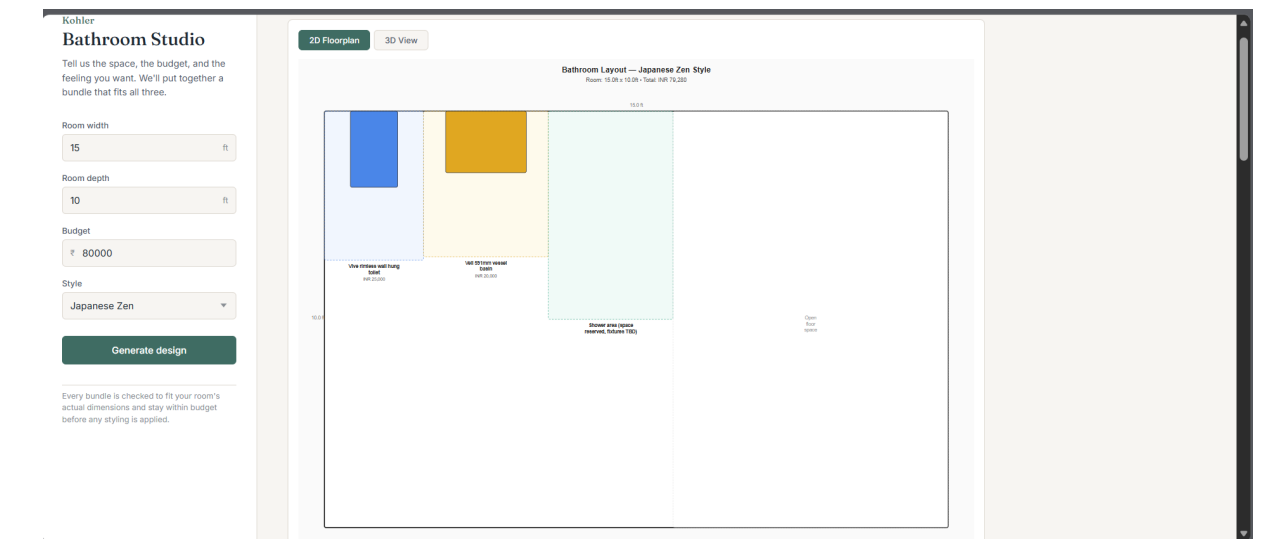
Prompt 6

```
C:\Users\Ira\Downloads\kohler-bathroom-ai (5)\kohler-bathroom-ai\src>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 113-602-776
127.0.0.1 - - [20/Sep/2026 14:10:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:10:20] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 14:10:20] "GET /static/vendor/three.min.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 14:10:20] "GET /static/vendor/GLTFLoader.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 14:10:20] "GET /static/vendor/OrbitControls.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Sep/2026 14:10:20] "GET /static/app.js HTTP/1.1" 200 -
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend
127.0.0.1 - - [20/Sep/2026 14:10:32] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:10:41] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:10:57] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:11:30] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:11:46] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:12:00] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:12:02] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:12:07] "POST /api/design HTTP/1.1" 200 -
127.0.0.1 - - [20/Sep/2026 14:12:49] "POST /api/design HTTP/1.1" 200 -
```

Prompt 7

- Uploaded image 1: screenshot — 9ft × 9ft room, “Modern” style, 2D floorplan (Total: INR 79,280)
- Uploaded image 2: screenshot — 15ft × 10ft room, “Japanese Zen” style, 2D floorplan (Total: INR 79,280)





Prompt 8

■ *Uploaded file: excerpt_from_previous_claude_message.txt — “To also dig into making different styles actually pick visually different products?”*

this i wanted to see a different visual for different style is that possible

Prompt 9

■ *(Selected from a multiple-choice prompt offering three options for how to add real style differentiation to the catalog.)*

Honest option: only use attributes already true in the real data (e.g. Span Round vs Span Square toilets, vessel vs wall-mount basins, existing mirror finishes) to write accurate style_tags — no invented products/finishes

Prompt 10

■ *Uploaded file: kohler_stageA_spatial_catalog_38_style_edition.json (a rebuilt product catalog with real per-product style_scores)*

i created the json file with different styles

Prompt 11

there isnt much change but can u make the ui interface for designer and interactive and uk better

Prompt 12

give me a overview and also tell what are the inputs what we have considered as fixed like the room layout and whatever we have changes give overview of it

Prompt 13

export the actual prompts of this conversation in a pdf

Prompts & AI Workflow Documentation

Kohler AI Bathroom Designer & Planner — Track 1, KOHLER-MITWPU AI Research Lab Program

This document records every AI prompt, system instruction, and AI-assisted workflow decision used to design and build this project — both the prompts running inside the application at runtime, and the prompts used to develop the application itself.

1. Overview: how AI was used in this project
2. Runtime prompt: Stage B aesthetic reasoning (Gemini)
3. Development workflow: architecture decisions made with AI assistance
4. Real bugs found and fixed through AI-assisted debugging
5. Data enrichment prompts (catalog construction)
6. Design-system prompts (UI/visual identity)
7. Explored-but-not-adopted: AI mesh generation (TripoSR)
8. Deliberate non-use of AI: where and why

1. Overview: how AI was used in this project

This project used AI in two distinct ways, which this document keeps clearly separated:

- **Runtime AI** — the one place the deployed application itself calls an LLM: Stage B's aesthetic reasoning, via the Google Gemini API. This is documented in Section 2 with the exact system prompt used.
- **Development-time AI assistance** — Claude (Anthropic) was used throughout the build process as a design and coding collaborator: architecture decisions, debugging, code review, and UI design. Sections 3-8 document this workflow, including the real mistakes caught and fixed along the way.

CORE ARCHITECTURAL PRINCIPLE (ESTABLISHED EARLY, HELD THROUGHOUT)

A single end-to-end LLM call cannot reliably do precise arithmetic (budget totals) or spatial reasoning (does this fit the room), and can hallucinate products that don't exist. The system is therefore split so that deterministic code enforces every hard constraint, and the LLM is only ever asked to make aesthetic judgments over a pre-validated set of options. This principle shaped every subsequent AI-related decision in the project.

2. Runtime prompt: Stage B aesthetic reasoning (Gemini)

This is the only prompt the live application sends to an LLM. It runs once per design generation. Conversational refinement is handled separately by deterministic code (see Section 8), not by re-calling this prompt. Source:

`src/stage_b.py` .

Design constraint on this prompt

The prompt deliberately excludes price and dimension data. Stage A has already produced 5 complete, budget-valid, spatially-valid bundles; Stage B's LLM call is not shown the numbers behind that validation at all — only product name, finish, material, and style tags. This makes it structurally impossible for the LLM's output to violate a hard constraint, regardless of how it reasons.

System prompt (verbatim)

SYSTEM

You are a bathroom design consultant for Kohler.

You will be given a style theme and a list of pre-validated, complete bathroom product bundles. Every bundle listed is ALREADY confirmed to fit the customer's room and budget - you do not need to check space or price.

Your only job:

1. Choose the ONE bundle (by its index) that best matches the given style theme, based on finish, material, and overall design coherence across the products in that bundle.
2. Write a short, customer-facing explanation (3-5 sentences) of why this bundle works well together for that style - mention specific finishes or materials that reinforce the coherence.

Rules:

- You MUST choose one of the bundle indexes given. Never invent a new combination or suggest a product that isn't already in the chosen bundle.
- Do not comment on price or whether something "fits" - that has already been verified. Focus purely on aesthetic and material coherence.
- Respond ONLY with valid JSON matching this exact schema, no other text:

```
{
  "selected_bundle_index": <integer>,
  "rationale": "<string>"
}
```

User prompt (templated, built per request)

USER

```
{
  "style_theme": "Modern",
  "bundles": [
    {
      "index": 0,
      "products": {
        "washbasin": {"name": "...", "finish": "...", "material": "...", "style_tags": [...]},
        "toilet": {...}, "shower_fixture": {...}, "bathroom_faucet": {...}, "mirror": {...}
      }
    },
    ... 4 more bundles ...
  ]
}
```

Response validation (defensive parsing)

The response is parsed defensively rather than trusted blindly:

- Markdown code-fence wrapping is stripped before JSON parsing, since LLMs sometimes add this despite instructions.
- The returned `selected_bundle_index` is validated against the actual number of bundles offered — an out-of-range or non-integer index is rejected and reported as an error, not trusted.
- If the API call fails for any reason (missing key, network error, malformed response), the system falls back to Stage A's top-quality bundle by score, with a clear on-page notice — the application never crashes on an LLM failure.

3. Development workflow: architecture decisions made with AI assistance

The following decisions, developed through discussion with Claude, shaped the core architecture, in the order they actually happened during development.

3.1 — The foundational architecture decision

Early in development, the naive approach ("send room + budget + style to an LLM and ask for a recommendation") was explicitly rejected after this reasoning was worked through with Claude:

DECISION

Split into Stage A (deterministic constraint filtering + budget optimization) and Stage B (LLM aesthetic judgment only, on a pre-validated shortlist). This was identified as the key differentiator for the "Approach & Innovation" evaluation criterion (45% weight) before any code was written.

3.2 — Why 5 bundles, not 1

The initial design had Stage A return a single "best" bundle by quality score. This was revised after discussion identified a risk: if Stage B had to assemble a bundle from per-category shortlists itself, it would need to re-verify budget math across categories — reintroducing the exact LLM-arithmetic risk the architecture was designed to avoid. Stage A was changed to return the **top 5 complete, pre-validated bundles**, so Stage B only ever chooses between whole valid options, never assembles one.

3.3 — Room model scope decision

A single-wall linear layout (toilet, vanity, shower arranged along one wall) was deliberately chosen over attempting full 2D bin-packing for arbitrary room shapes. This was an explicit scope decision, documented in the README as a known MVP limitation rather than an oversight — full geometric layout optimization was judged to cost disproportionate development time versus its likely impact on evaluation.

4. Real bugs found and fixed through AI-assisted debugging

These are genuine defects caught during development — most by actually running the code and inspecting output, not by inspection alone. Documenting them honestly, since catching and fixing real bugs is itself part of the AI-assisted engineering workflow.

BUG 1 — CASE-SENSITIVE STYLE MATCHING

`style in product["style_tags"]` did an exact string match, so a user typing "modern" (lowercase) silently failed to match a catalog tag of "Modern". Found by testing the CLI with lowercase input and observing the style bonus wasn't applied. Fixed with `.lower()` normalization on both sides.

BUG 2 — STAGE B CLIENT CREATION OUTSIDE TRY/EXCEPT

`genai.Client()` was called outside the function's error-handling block, so a missing API key crashed the whole request with a raw exception instead of triggering the intended graceful fallback to Stage A's top bundle. Found by testing the web UI without an API key set. Fixed by moving client construction inside the `try` block.

BUG 3 — CSS SPECIFICITY HIDING THE EMPTY STATE INCORRECTLY

`.result { display: grid }` overrode the browser's default `[hidden] { display: none }` behavior, so "empty" result panels were visible before any design had been generated. Found via a Playwright screenshot of the page's initial state. Fixed with an explicit `[hidden] { display: none !important; }` rule.

BUG 4 — HTML5 VALIDATION SILENTLY BLOCKING FORM SUBMISSION

A budget input with `min="1" step="500"` rejected the default value `80000` as "invalid" (not reachable via 500-increments from 1), silently blocking submission with no visible error. Found via a real user test run. Fixed by simplifying to `step="1"`.

BUG 5 — 3D BASIN RENDERING AS A FLAT DISC, NOT A BOWL

A procedural basin shape applied local non-uniform flattening to a sphere *in addition to* the parent group's real-dimension scaling — since a basin's real height is naturally much smaller than its width, the two flattenings compounded into an unrecognizable pancake shape. Found by visually inspecting a rendered screenshot. Fixed by removing the redundant local scale and letting the real catalog dimensions shape the sphere naturally.

BUG 6 — STRAY CORRUPTED CODE IN STAGE_D.PY

An unrelated, never-authored function (`apply_direct_selection`) was found merged into the refinement module, corrupting the real function's definition line and causing a Python `IndentationError`. Root cause was file-system contamination in the development sandbox, not a logic error. Found by running `python -m py_compile` across every file after an unrelated change — now a standard pre-commit check for this project.

5. Data enrichment prompts (catalog construction)

The product catalog (`data/kohler_catalog.json`) was built iteratively, combining real Kohler India product data with AI-assisted enrichment for fields not published per-SKU. The general pattern used for enrichment prompts:

```
Generate [missing field] for these [N] real Kohler products, using  
[documented industry standard / cited source] where exact SKU-level  
data is not available. Tag each value with a confidence level  
(high/medium/low) and a one-line source explanation. Do not invent  
SKU-specific facts - only apply category-level standards explicitly  
and transparently.
```

This pattern was used to derive: spatial clearance rules (sourced from NKBA — National Kitchen & Bath Association — planning standards), category-level material inference (e.g., toilets/basins as vitreous china, mirrors as glass + metal frame), and finish-code decoding (e.g., `CP` → Polished Chrome). Every enriched field carries a source and confidence tag in the catalog JSON, so real data and documented-standard fallbacks are never conflated.

6. Design-system prompts (UI/visual identity)

The web interface's visual design was deliberately directed away from generic "AI tool" defaults. The guiding brief given to Claude for the interface:

Design a "design consultation" layout, not a SaaS dashboard: a narrow spec-sheet input rail, a large results canvas. Avoid the generic warm-cream-and-terracotta AI palette - use a palette tied to the subject matter (water, bath, real Kohler finish names). Pair an editorial serif headline font with a clean sans body font, not a single system font. Flat panels with hairline borders, not shadow-card kits.

This produced the Porcelain/Teal/Brass palette and the Fraunces/Inter type pairing used throughout the application and this document.

7. Explored-but-not-adopted: AI mesh generation (TripoSR)

A real, substantial engineering path was explored to generate actual AI-reconstructed 3D meshes (`.glb` files) from product photos, using TripoSR (single-image-to-3D reconstruction), for the 3D view. This is documented honestly here because it represents genuine AI-assisted engineering work, even though the final submission defaults to procedural shapes.

ROUTE ATTEMPTED	OUTCOME
Local TripoSR (Windows)	Succeeded — required installing a correct 64-bit MSVC toolchain (Visual Studio Build Tools), fixing a <code>torchmcubes</code> build-isolation issue (<code>--no-build-isolation</code> flag), a missing <code>pybind11</code> CMake path, and a 32-bit/64-bit architecture mismatch (wrong compiler variant initially selected)
Google Colab (cloud GPU/CPU)	Not pursued to completion — local route succeeded first
Hugging Face hosted demo	Not pursued — blocked by a real, external platform outage at the time of initial investigation
Replicate API (paid)	Not pursued — ruled out on cost grounds before the free local route was retried and succeeded

FINAL DECISION

A three-tier fallback system was built: real AI-generated mesh (if a `.glb` is present) → procedural shape (the guaranteed, always-available fallback). The submission ships with the procedural tier only, by deliberate choice — the AI-mesh integration code is complete and tested, but generating meshes for the full product set was judged not worth the remaining time before the deadline. Single-photo 3D reconstruction also has a genuine quality ceiling (rough, approximate geometry) that was weighed honestly against the guaranteed reliability of procedural shapes.

8. Deliberate non-use of AI: where and why

Two components deliberately do *not* call an LLM, by explicit design choice rather than oversight:

8.1 — Stage A (constraint engine): no AI

Every spatial-fit and budget check is plain, deterministic Python. This is the component whose output must never be wrong, so it uses code, not model inference, for every decision.

8.2 — Stage D (conversational refinement): rule-based, not a second LLM call

When a user types "show me a cheaper vanity," the system parses this with keyword matching rather than a second Gemini call. This was a deliberate reliability decision:

Intent parsing here is deliberate rule-based keyword matching, not another LLM call. This keeps refinement instant, free, and immune to API failures - a second network-dependent AI call would add risk without adding real value, since the vocabulary space for this kind of request is small and well-defined.

The actual product-swap decision that follows is also 100% deterministic code, selecting only from products already proven to fit the room and stay in budget — the same "hard rules over LLM guesswork" principle applied consistently across the whole system.